

Farmer Marketplace: Matching, Scoring and Pooled Transport

How a buyer request finds nearby farms, how candidates are scored and ranked, and how compatible orders are pooled into one shipment. All numbers come from the seeded demo data and the code in `api/app/matching` and `api/app/logistics`.

1. Algorithms at a glance

Step	Algorithm	Where
Find nearby farms (buyer)	PostGIS ST_DWithin radius filter, ST_Distance on WGS84 geography (metres)	<code>matching/engine.py</code>
Score a candidate	Weighted linear score, lower is better	<code>matching/engine.py</code>
Rank and fill the order	Sort by score, then greedy fill until the quantity is met (multi-farmer aggregation)	<code>matching/engine.py</code>
Distance inside transport pools	Haversine great-circle distance x 1.3 road factor (mock routing provider)	<code>logistics/routing.py</code>
Group orders into pools	Greedy single-pass clustering on destination, window, pickup proximity, capacity	<code>logistics/pooling.py</code>
Order the stops	Nearest-neighbour heuristic: pickups first, then deliveries	<code>logistics/pooling.py</code>
Decide if a pool is worth it	Cost comparison, pool only if it saves at least 10%	<code>logistics/pooling.py</code>

2. Finding nearby farms for a buyer

A buyer request stores a pickup point and a search radius (default 50 km). Farms and requests are stored as PostGIS **geography** points (SRID 4326), so distances are computed on the WGS84 ellipsoid in metres, not on a flat plane.

The query keeps only lots that pass all of these filters:

```
active lot
AND lower(product) == lower(request.product)
AND ST_DWithin(farm.location, request.location, radius_km * 1000)
AND price_per_kg <= request.max_price_per_kg  (when a max price is set)
```

ST_DWithin uses the GiST spatial index on `farms.location`, so the radius filter does not scan every farm. **ST_Distance** then returns the exact distance in metres for each survivor, converted to km for scoring. This is straight-line (geodesic) distance. It is not road distance, because buyer search only needs to rank farms by proximity. Road distance is introduced in transport pooling (section 4).

Why not DFS or a graph search? DFS answers reachability, not weighted shortest paths. A proximity search is a spatial index lookup, and a true road shortest path (Dijkstra or A*) belongs in a routing engine such as OSRM or GraphHopper behind the `RoutingProvider` interface.

3. How the score works

Each eligible lot gets one explainable number. **Lower score = better match**. There is no machine learning at this stage.

$$\text{score} = 0.5 * \text{distance_score} + 0.3 * \text{price_score} + 0.2 * \text{fulfilment_risk}$$

$\text{distance_score} = \text{distance_km} / \text{radius_km}$ 0 = at the buyer, 1 = at the edge
 $\text{price_score} = \text{price_per_kg} / \text{price_ceiling}$ ceiling = buyer max price, or the highest eligible price
 $\text{fulfilment_risk} = 1 - \min(\text{lot_qty} / \text{request_qty}, 1)$ 0 = one lot covers it all

distance 50%	price 30%	fulfilment risk 20%
--------------	-----------	---------------------

The risk term favours lots that can cover most of the request alone: fewer farmers means fewer things that can fail. Candidates are sorted by (score, distance, price), so ties break deterministically on the closest then cheapest lot.

Worked example: buyer request for 100 kg of tomatoes, max price 0.80/kg, radius 50 km

	Green Valley (lot_demo_tomatoes)	Sunrise Fields (lot_demo_tomatoes_sunrise)
Available	60 kg at 0.70/kg	50 kg at 0.75/kg
Distance	0.90 km	1.39 km
distance_score	$0.90 / 50 = 0.0180$	$1.39 / 50 = 0.0278$
price_score	$0.70 / 0.80 = 0.8750$	$0.75 / 0.80 = 0.9375$
fulfilment_risk	$1 - 60/100 = 0.4000$	$1 - 50/100 = 0.5000$
score	$0.5(0.0180) + 0.3(0.8750) + 0.2(0.4000) = 0.3515$	$0.5(0.0278) + 0.3(0.9375) + 0.2(0.5000) = 0.3952$
Rank / allocation	#1: takes 60 kg	#2: takes the remaining 40 kg

Greedy fill: walk the ranked list and take $\min(\text{remaining}, \text{lot_quantity})$ from each lot until nothing remains. Here $60 + 40 = 100$ kg, a complete match. If supply runs out first the plan is marked *partial* and the buyer sees the shortfall; a grouped order can only be created from a complete plan. Greedy fill is optimal for the ranking given and cheap to explain. It is not guaranteed to minimise total cost across every combination of farms.

4. Transport pools

After a farmer accepts an order, the marketplace can combine compatible orders into one shipment to cut delivery cost. Pools are proposed by a transporter or admin, then confirmed by a transporter with a vehicle quote.

Distance model

Pool costs need distances between farms and delivery points. The default MockRoutingProvider uses the **Haversine** great-circle formula multiplied by a **1.3 road factor** to approximate winding roads:

```

a = sin^2(dlat/2) + cos(lat1) * cos(lat2) * sin^2(dlon/2)
km = 2 * 6371 * asin(sqrt(a)) * 1.3

route(stops) = sum of estimate(stop_i, stop_i+1)
  
```

If the primary routing provider throws (for example OSRM is down), FallbackRoutingProvider silently uses the mock estimate, so pooling never fails because a map service is offline. Replace the mock with OSRM or GraphHopper for real road distances. The pooling code does not change.

Step 1: which orders can pool together (greedy clustering)

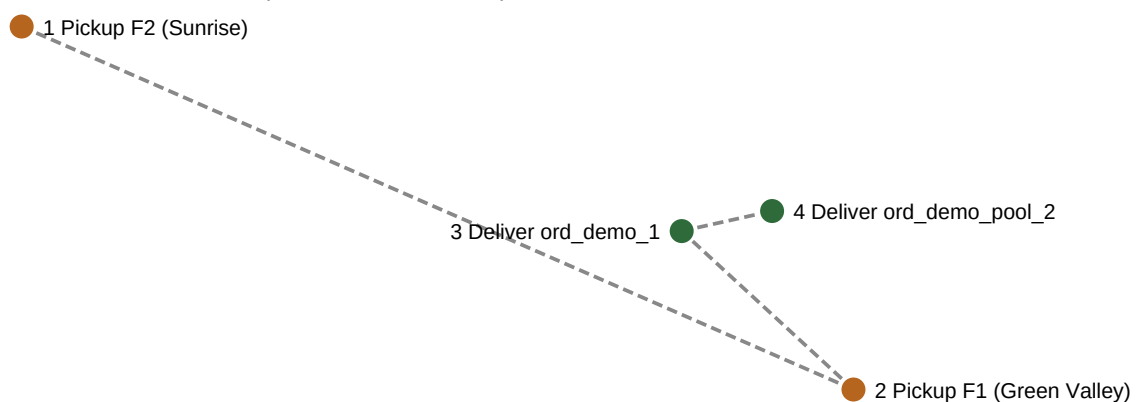
Eligible orders are **ACCEPTED** or **READY_FOR_PICKUP**, have a delivery point, and are not already in a pool. Orders are sorted by id, then each is placed into the first cluster where **all** hold:

Rule	Value
Destination within	5 km of the cluster's first destination
Pickup within	30 km of the cluster's first pickup
Delivery windows	Overlap (an open window matches anything); intersection must not be empty
Total weight	At most the largest active vehicle's capacity (1000 kg if no vehicles are registered)

Step 2: order the stops (nearest neighbour)

Start at the pickup farthest from the destination, repeatedly drive to the nearest unvisited pickup, then do the same for the deliveries starting from the last pickup. This is a classic heuristic: fast and predictable, but not always the shortest possible tour.

Pooled route for the demo orders (schematic, not to scale)



Step 3: pool only if it saves money

```

cost(km)          = base_fee + cost_per_km * km          default 10.00 + 0.80/km
separate_cost     = sum over orders of cost(route[pickup, destination])
pooled_cost       = cost(route[all pooled stops in order])
  
```

```

create the pool only if  pooled_cost <= separate_cost * (1 - 0.10)
  
```

Worked example from the demo data (ord_demo_1 20 kg and ord_demo_pool_2 10 kg)

Item	Calculation	Result
Order 1 alone	Green Valley to buyer: 1.17 km, 10 + 0.8 x 1.17	10.94
Order 2 alone	Sunrise to buyer: 1.79 km, 10 + 0.8 x 1.79	11.43
Separate total	10.94 + 11.43	22.37
Pooled route	Sunrise to Green Valley 2.93 + to deliver 1 1.17 + to deliver 2 0.20	4.31 km
Pooled cost	10 + 0.8 x 4.31	13.45
Savings	22.37 - 13.45 = 8.92, above the 10% threshold	39.9%

Vehicle quotes

For every active vehicle whose capacity fits the pool's total weight, a quote is created using that vehicle's own rates: $\text{base_fee} + \text{cost_per_km} * \text{route_km}$. The demo van (12.00 base, 0.90/km) quotes 12 +

$0.9 \times 4.31 = \mathbf{15.88}$. Only the vehicle's own transporter can confirm a quote. Confirming assigns the vehicle to the shipment, and only then can that transporter mark pickup and delivery.

Order lifecycle used by transport

PENDING -> ACCEPTED -> READY_FOR_PICKUP -> IN_TRANSIT -> DELIVERED -> COMPLETED

PENDING to ACCEPTED farmer accepts
ACCEPTED to READY_FOR_PICKUP farmer marks the produce ready
READY_FOR_PICKUP to IN_TRANSIT transporter confirms pickup (confirmed shipment only)
IN_TRANSIT to DELIVERED transporter confirms delivery
DELIVERED to COMPLETED buyer confirms receipt

5. Known limits and upgrade paths

Limit	Upgrade path
Buyer search ranks by straight-line distance, not road distance	Re-rank the survivors with a RoutingProvider (OSRM/GraphHopper)
Fixed score weights 50/30/20	Make them configurable per request once real fulfilment data exists
Greedy fill is not globally cost-optimal	Solve allocation as an optimisation problem (OR-Tools) if cost matters more than explainability
Mock routing uses a fixed 1.3 road factor	Real routing engine behind the same interface
Greedy clustering and nearest-neighbour stops	OR-Tools capacitated vehicle routing with time windows when data shows the heuristic is insufficient
Orders that cannot pool get no shipment	Create single-order shipments when needed